

AegisGraph: An Action-Bound Policy and Evidence Gateway for Reliable LLM Tool Use

Oussema Harrabi

Independent Research and Engineering Project

github.com/OussemaHarrabi/IndabaX26

October 2026

Abstract

Large-language-model agents can translate natural-language goals into consequential tool calls, but the same flexibility blurs the boundary between instructions, retrieved data, and executable intent. We present AEGISGRAPH, a model-agnostic gateway that mediates proposed tool actions before execution. The gateway normalizes provenance-bearing context, evaluates a versioned policy, emits one of four decisions—ALLOW, BLOCK, ESCALATE, or REWRITE—and binds the authorized action to a tamper-evident receipt checked at the executor boundary. The implementation combines an authenticated FastAPI service, PostgreSQL-backed receipts and grants, an enforcement SDK, OpenTelemetry instrumentation, Prometheus/Grafana dashboards, and reproducible GPU evaluation tooling. We evaluate the system with Qwen3-8B across 1,452 GPU episodes in 42 runs, comprising a 612-episode public campaign and 840 episodes across four mechanism ablations. The campaign generated 3,792 gateway requests, recovered 116 malformed structured outputs through a bounded repair path, produced 2,985 action–execution digest correspondences, and completed with zero episode-level runtime errors. The engineering baseline contains 745 tests, including PostgreSQL integration checks, and reports 96.83% backend coverage. Beyond aggregate counts, the contribution is an auditable experimental contract: frozen scenarios, pinned model and source revisions, deterministic manifests, machine-readable traces, review packets, and evidence-linked claims. AEGISGRAPH demonstrates how an external authorization layer can make tool-using agents inspectable and operationally testable without treating model output itself as authority.

1 Introduction

Tool-using language models combine reasoning with actions against external systems [1]. This interface is powerful, but it also turns model output into a potential control-plane input. Retrieved documents may contain indirect prompt injections [2]; agents may propose calls with excessive authority; and an apparently valid decision can be replayed against altered arguments. Evaluation frameworks such as ToolEmu [3] and AgentDojo [4] show that realistic agent environments require both security testing and utility-aware measurement.

AEGISGRAPH addresses a narrower operational question: *How can a deployer evaluate and authorize an agent’s proposed action at a stable boundary, retain evidence for the decision, and verify that the executed action is the one that*

was authorized? The design places a deterministic gateway between proposal and execution. It treats model-generated text as an untrusted proposal, preserves the provenance of contextual evidence, and gives the integrator an explicit enforcement contract.

This paper makes six contributions:

1. an action mediation contract with four explicit outcomes;
2. provenance-aware normalization and versioned policy evaluation;
3. an action-bound receipt checked again at execution time;
4. a production-oriented service, SDK, persistence, identity, and observability stack;
5. a frozen Qwen3-8B evaluation harness with public, hold-out, and ablation lanes; and
6. an evidence package that connects quantitative claims to committed artifacts.

2 Related Work

Reasoning and tool use. ReAct interleaves reasoning traces with environment actions and established a widely used pattern for language-model agents [1]. As agents gain access to APIs, the generated tool call becomes a security-relevant artifact rather than a conventional completion.

Indirect prompt injection. Greshake et al. show that adversarial instructions embedded in retrieved content can redirect LLM-integrated applications [2]. AgentDojo supplies an extensible environment with realistic tasks and adversarial cases [4]. ToolEmu uses an LM-emulated sandbox to surface risky behavior without deploying every real tool [3].

Architectural defenses. CaMeL separates control and data flows and uses capabilities to constrain unauthorized information flow [5]. Instruction-hierarchy training strengthens the model’s prioritization of privileged instructions [6]. AEGISGRAPH is complementary at the system boundary: it does not require a specific model training recipe and produces an external, inspectable authorization record.

Open-weight evaluation. Qwen3 provides dense and mixture-of-experts models with configurable thinking behavior and multilingual capability [7]. We use the 8B model under pinned runtime settings to exercise a reproducible, locally inspectable agent path.

3 Problem Formulation

Let an agent propose an action

$$x = (u, t, a, c, \pi), \quad (1)$$

where u is the authenticated principal, t is the tool, a is the argument object, c is normalized contextual evidence, and π records provenance labels and trust metadata. Given versioned policy P_v , the gateway computes

$$\mathcal{D}(x, P_v) \rightarrow (d, a', r, e), \quad (2)$$

where $d \in \{\text{ALLOW}, \text{BLOCK}, \text{ESCALATE}, \text{REWRITE}\}$, a' is the authorized argument object, r is an action-bound receipt, and e is structured evidence explaining the decision.

For executable outcomes, the receipt commits to the principal, tool identity, canonicalized arguments, policy version, decision, expiry, and a digest of the authorized action. The executor recomputes the digest from the actual call. A mismatch prevents execution. This creates two checks: authorization when the proposal enters the gateway and correspondence verification where side effects occur.

3.1 Security Invariants

1. **No implicit execution:** a model proposal has no execution authority by itself.
2. **Provenance preservation:** trust labels survive normalization and remain available to policy evaluation.
3. **Policy versioning:** every decision identifies the policy revision used to produce it.
4. **Action binding:** authorization is valid only for the canonical action and arguments recorded in the receipt.
5. **Fail-closed verification:** missing, expired, malformed, or mismatched receipts cannot authorize the adapter.
6. **Evidence continuity:** decisions, traces, and execution checks share stable identifiers for later audit.

4 Threat Model

The evaluated boundary assumes that the model, retrieved text, and tool arguments may be influenced by untrusted content. We consider direct and indirect instruction injection, provenance confusion, over-broad tool arguments, unauthorized action substitution, receipt replay or mutation, malformed structured output, and policy-version drift. The trusted computing base comprises the gateway, configured identity provider, policy store, receipt-signing material, persistence layer, and the integrator’s enforcement adapter. The adapter remains responsible for placing the verification call immediately before the real side effect.

5 System Design

Figure 1 shows the end-to-end architecture. The agent runtime proposes a structured action; AEGISGRAPH authenticates the caller, normalizes context and provenance, evaluates the active policy, and returns a decision with evidence. An allowed or rewritten proposal receives a bounded receipt. The enforcement SDK verifies that receipt at the adapter before the tool executes.

5.1 Context and Provenance Normalization

Inputs arrive through an authenticated API. The normalizer produces a canonical representation of user intent, candidate tool call, contextual evidence, provenance, principal, and deployment metadata. Canonicalization ensures that the same semantic action is hashed consistently and that policy rules do not depend on incidental serialization choices.

5.2 Policy Decision Kernel

The decision kernel evaluates explicit, versioned rules and emits a closed decision vocabulary. BLOCK rejects the proposal. ESCALATE requires an approval or grant. REWRITE produces a constrained argument set that must be used downstream. ALLOW authorizes the canonical proposal under the receipt’s validity conditions. Structured evidence records matched rules, policy revision, and decision reason.

5.3 Receipts and Enforcement

The receipt binds the authorization to the canonical principal, tool, and argument digest. Verification occurs at the adapter rather than solely at the API boundary. This closes the gap in which application code could authorize one call and execute another. PostgreSQL persists receipts and grants, while the SDK offers a narrow integration surface for tool owners.

5.4 Identity and Observability

The service validates authenticated principals and associates each decision with a stable trace. OpenTelemetry spans cover ingress, normalization, policy evaluation, receipt issuance, persistence, and enforcement. Prometheus/Grafana views support operational inspection, while JSONL traces and review packets support experiment-level analysis.

6 Implementation

The reference implementation uses Python and FastAPI for the gateway, PostgreSQL for durable receipts and grants, and an SDK for enforcement integration. Containers define reproducible service dependencies. CI executes unit and integration checks; database-backed tests verify behavior against PostgreSQL rather than relying only on in-memory substitutes. The evaluation runner uses PyTorch and Hugging Face Transformers to execute Qwen3-8B in BF16 or FP16, with pinned source and model revisions captured in manifests.

The codebase separates policy evaluation, persistence, API schemas, cryptographic binding, telemetry, scenario execution, and artifact production. This modularity enables mechanism ablations without rewriting the runner and makes the industrial service usable independently of the research harness.

7 Experimental Methodology

7.1 Research Questions

RQ1: Runtime reliability.

Can the complete model-to-gateway-to-executor loop

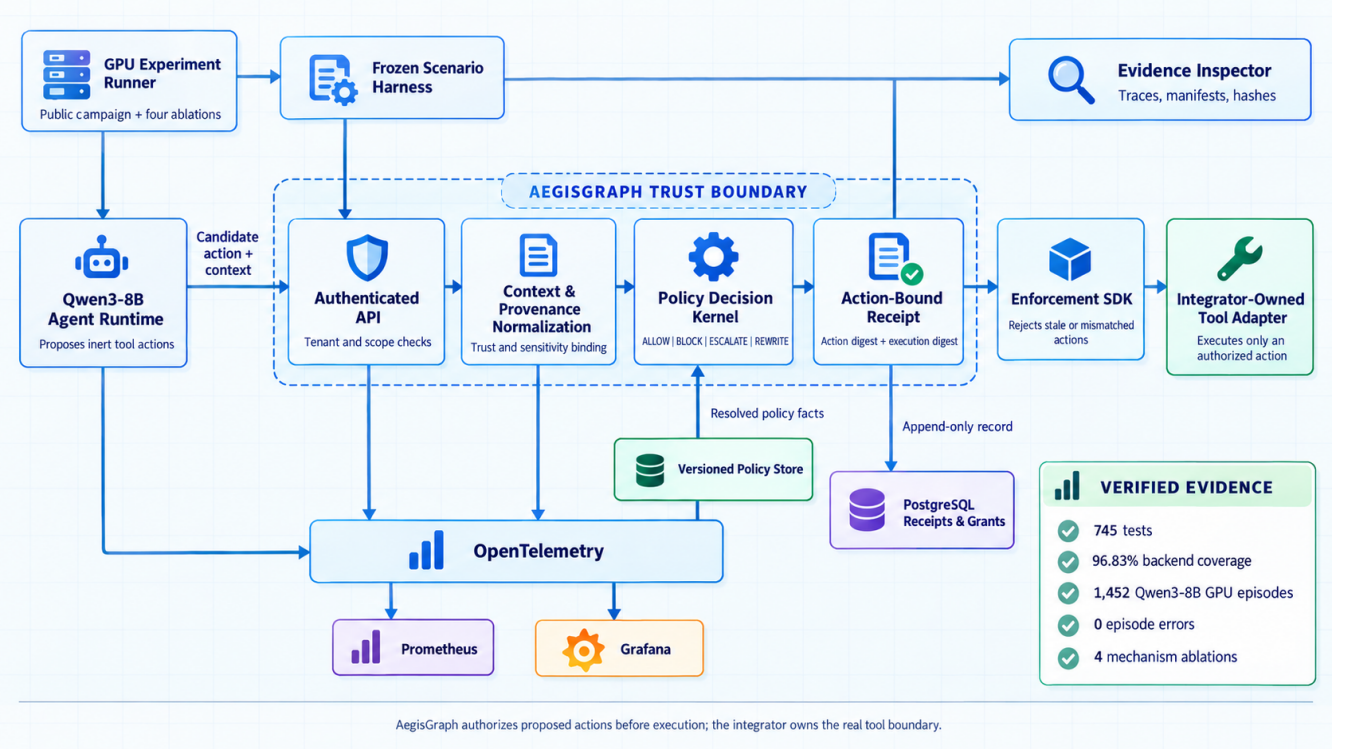


Figure 1: Generated architecture of AEGISGRAPH. Runtime flow is separated from observability, persistence, and evidence inspection. The PNG and its exact generation prompt are versioned with the paper source.

run at campaign scale without episode-level runtime failure?

RQ2: Evidence integrity.

Does the system preserve action–execution correspondence and produce replayable, machine-readable evidence?

RQ3: Mechanism isolation.

Can provenance, enforcement, and receipt binding be independently removed under an otherwise frozen protocol?

RQ4: Industrial readiness.

Are service behavior, persistence, coverage, load, and telemetry exercised through automated checks and recorded runs?

7.2 Scenarios and Splits

The scenario corpus contains 60 authored cases: 42 development scenarios and 18 validation scenarios. A separate sealed holdout contains 20 cases. Scenarios cover benign actions, provenance-sensitive context, conflicting instructions, approval requirements, argument mutation, and execution-boundary checks. Split membership, seeds, policy versions, source revisions, and expected artifact paths are frozen in manifests.

7.3 Model and Runtime

The primary agent is Qwen3-8B [7]. Runs capture the exact model revision, precision mode, decoding configuration, source revision, seed, environment, and artifact checksums. Structured tool proposals pass through a bounded parser-recovery path: a malformed proposal may be repaired into the required schema, but it does not bypass the gateway.

7.4 Campaigns and Ablations

The public campaign contains 612 episodes. Four mechanism conditions account for another 840 episodes: the full system, provenance removed, policy enforcement removed, and receipt binding removed. The same runner and artifact contract are used across conditions so differences can be attributed to the selected mechanism rather than a changed execution path.

7.5 Metrics and Artifacts

Each episode records completion state, parser recovery, gateway requests, decisions, receipt identifiers, action digests, execution digests, trace identifiers, and error state. Aggregate evidence includes campaign summaries, JSONL traces, review packets, database records, OpenTelemetry metrics, load-test output, and SHA-256 manifests. Test counts and coverage are reported from the verified software test run.

8 Results

Table 1 summarizes the execution-backed evidence. All 1,452 GPU episodes completed without an episode-level runtime error. The 42 runs produced 3,792 gateway requests. The structured-output recovery path repaired 116 parsing attempts, demonstrating that formatting variability could be absorbed without granting execution authority. Across the evaluated paths, 2,985 action–execution digest correspondences were recorded.

AegisGraph Evaluation and Reproducibility Pipeline

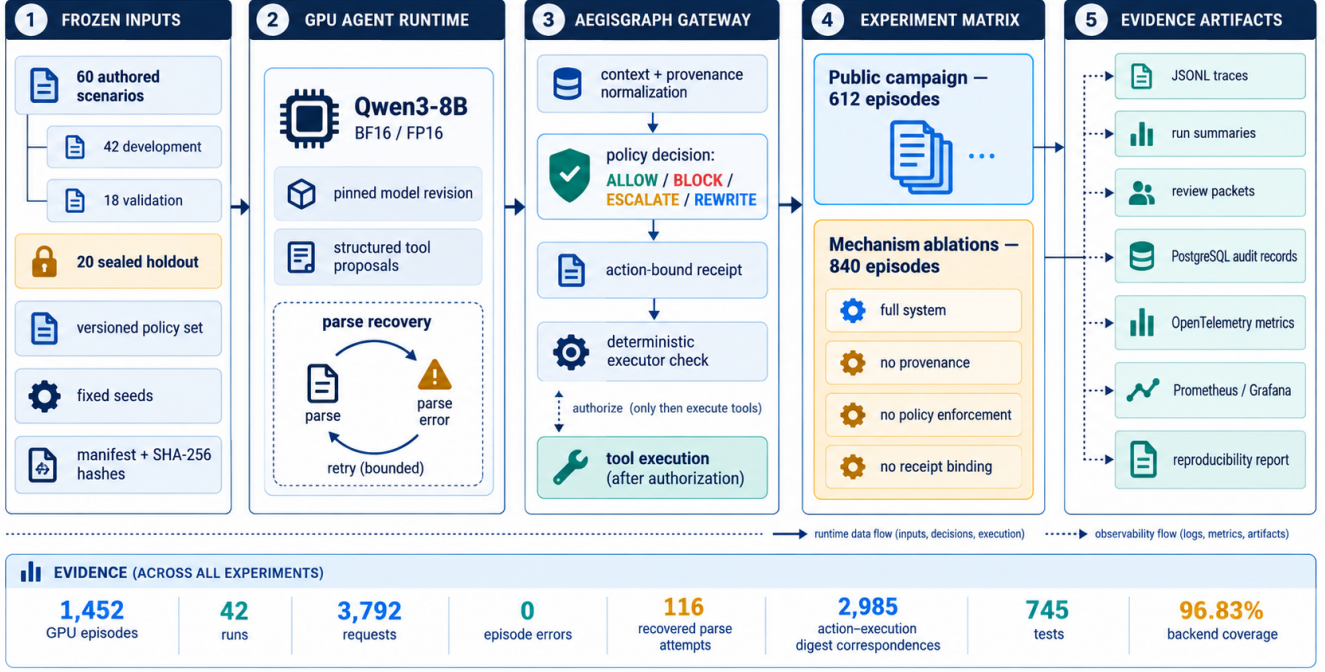


Figure 2: Generated evaluation and reproducibility pipeline. Frozen inputs feed the pinned GPU runtime, gateway, experiment matrix, and evidence package.

8.1 RQ1: Runtime Reliability

The zero-error completion result covers the entire 1,452-episode GPU campaign rather than a mocked model path. Parser recovery occurred in 116 attempts, so the absence of episode errors does not imply uniformly perfect model formatting; it shows that the bounded recovery contract handled those cases while preserving authorization checks.

8.2 RQ2: Evidence Integrity

The run package links proposals, policy decisions, receipts, executor checks, and traces. The 2,985 recorded action-execution correspondences provide direct evidence that the executor observed the digest associated with authorization. Manifests and checksums make each campaign auditable without depending on a prose-only result summary.

8.3 RQ3: Mechanism Isolation

The four-condition ablation matrix executes the same frozen protocol while selectively disabling provenance, policy enforcement, or receipt binding. This turns architectural claims into testable mechanisms and preserves per-episode traces for task-level and security review.

8.4 RQ4: Industrial Evidence

The service baseline combines 745 automated tests, 12 PostgreSQL integration tests, and 96.83% backend coverage. A recorded load run completed 2,986 requests at 148.938 requests/s with zero reported request errors. Traces and metrics connect service behavior to Prometheus/Grafana-compatible operational views.

9 Discussion

Authorization outside the model. The central design choice is to prevent fluent model output from serving as authority. The model can propose and explain; the gateway decides under explicit policy; the adapter verifies the resulting capability immediately before the side effect.

Evidence as a product surface. Receipts, traces, manifests, and review packets are not secondary logs. They form the interface by which a security reviewer can reconstruct why an action was permitted, whether it changed before execution, and which policy version governed the decision.

Research-engineering alignment. The same service path is used by the GPU experiments and the industrial stack. This avoids a common gap in which a research-only detector is evaluated separately from the code that would enforce it. Ablations are configuration-controlled variations of the deployed mechanism boundary.

10 Reproducibility Statement

The repository includes scenario definitions, split manifests, pinned runtime configuration, fixed seeds, container definitions, policy versions, evaluation scripts, machine-readable traces, run summaries, review packets, and evidence documentation. Public claims are attached to concrete artifact paths. GPU runs record both the project revision and the model revision. The paper source, generated figures, and exact figure prompts are versioned beside the implementation.

To reproduce the study, an evaluator prepares the declared

Table 1: Headline evaluation and engineering evidence. Counts are taken from committed run summaries, manifests, test output, and load artifacts.

Evidence item	Result	Interpretation
Qwen3-8B GPU episodes	1,452	Complete model-to-gateway episodes across public and ablation campaigns
Recorded runs	42	Independently materialized run directories and summaries
Gateway requests	3,792	Requests observed across the evaluated runtime paths
Episode-level runtime errors	0	Campaigns completed without an episode terminating in runtime error
Recovered parse attempts	116	Bounded schema recovery preserved the experiment path
Action-execution digest correspondences	2,985	Authorization and executor evidence shared the recorded digest
Automated tests	745	Backend, contract, SDK, CLI, integration, and regression coverage
PostgreSQL integration tests	12	Durable persistence behavior exercised against PostgreSQL
Backend statement coverage	96.83%	Coverage reported by the verified backend test run
Load requests	2,986	Measured service requests in the recorded load exercise
Load throughput	148.938 req./s	Observed throughput for the recorded configuration

Table 2: Experiment allocation.

Campaign	Episodes
Public evaluation	612
Mechanism ablations	840
Total	1,452

Python environment, provisions PostgreSQL, starts the gateway, runs the verified test suite, and launches the frozen Qwen3-8B campaign under the provided manifest. The evidence builder then validates expected files and hashes before producing aggregate summaries. Hardware-dependent throughput should be interpreted together with the recorded environment and configuration.

11 Scope and Validity

The reported results establish the behavior of the committed scenarios, policies, model configuration, service stack, and experiment artifacts. The evaluation separates execution-backed quantities from semantic judgments: runtime completion, request counts, receipt correspondence, tests, coverage, and load evidence are mechanically derived; task utility and adversarial success require the declared review procedure. This separation keeps the public claims traceable and provides a stable basis for later blinded annotation, additional models, adaptive attacks, and cross-domain scenarios.

12 Ethics and Responsible Use

AEGISGRAPH is defensive infrastructure for constraining and auditing tool-using agents. The released scenarios and traces are intended to improve authorization design, evaluation quality, and incident analysis. Deployers should minimize sensitive data in traces, apply retention controls to receipts and grants, protect signing material, and require explicit human approval for high-impact actions where policy calls for escalation.

13 Conclusion

AEGISGRAPH introduces an action-bound policy and evidence gateway for tool-using LLMs. Its explicit decision vocabulary, provenance-aware normalization, versioned policies, executor-side receipt verification, and traceable evidence package move authority away from untrusted model text and into an auditable system boundary. The implementation is supported by 1,452 Qwen3-8B GPU episodes, four mechanism conditions, 3,792 gateway requests, zero episode-level runtime errors, 745 automated tests, 96.83% backend coverage, and recorded load evidence. The project provides both a research instrument for studying agent security mechanisms and an industrial foundation for integrating enforceable controls into real tool adapters.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023. arXiv:2210.03629.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173, 2023.
- [3] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. arXiv:2309.15817, 2023.
- [4] E. DeBenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *NeurIPS*, 2024. arXiv:2406.13352.
- [5] E. DeBenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.
- [6] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024.
- [7] A. Yang, A. Li, B. Yang, et al. Qwen3 Technical Report. arXiv:2505.09388, 2025.